ジオメトリック・インテリジェンス

ケーススタディ補助文書 03

実装スクリプト集

― 経済安全保障多様体の構築に使われた Python 実装 ―

本書は、ケーススタディ本編「国家の曲率」に登場する Python コード断片を、
GI 理論のパイプラインの流れに沿って整理し、
各コード片の数学的背景、運用上の意図、実装上の注意を解説した実装ガイドである。

Étale Cohomology

2026 年 10 月

License: CC BY 4.0

目次

1　はじめに **2**

1.1　本書の位置づけ 2

1.2　掲載コードについて 2

1.3　本作のコードの特徴 2

1.4　推奨ライブラリ 2

2　多様体の構築 **3**

2.1　命題 1.1 の検証 — 第 3 回試行（合格） 3

2.2　データの匿名化（本編第 1〜2 章の設計） 3

3　リー微分とリー括弧 **4**

3.1　台湾海峡封鎖演習のリー微分 4

3.2　リー括弧 — 台湾封鎖とレアアース停止 5

3.3　三重リー括弧 — 台湾 × レアアース × 穀物 5

4　パーシステント・ホモロジーとゲーム理論 **6**

4.1　危機の点の集まりの形 6

4.2　国家間のナッシュ均衡 6

5　層ラプラシアン **7**

5.1　政府の組織の層ラプラシアン 7

6　手術とシンプレクティック積分 **8**

6.1　国家規模の手術計画 8

6.2　15 年間の航海図 9

7　ZKML **10**

7.1　首相への最終証明 10

付録 A　参照実装 **11**

第1章　はじめに

1.1　本書の位置づけ

本書は「国家の曲率」の補助文書 03 である。各節の「参照」では、補助文書 02 の章と、GI 理論のコンパニオン（Vol.1 / Vol.2）の該当章を示す。コンパニオンは次の URL で公開されている。

- Vol.1：https://github.com/EtaleCohomology/geometric-intelligence-saga/blob/main/companion/GI_Companion_Vol1_JP.pdf
- Vol.2：https://github.com/EtaleCohomology/geometric-intelligence-saga/blob/main/companion/GI_Companion_Vol2_JP.pdf

1.2　掲載コードについて

本書に掲載したコード断片は、本編に登場したコードを再録したものである。本編のコードは、物語の中で数式とコードを橋渡しすることを優先しているため、そのままでは実行できない箇所（外部関数の呼び出し、省略された処理、本編の数値を出力として示した部分など）を含む。教育目的の簡略化のために数学的に厳密でない箇所もあり、本書ではそれらを各節の「実装上の注意」で指摘し、正しい実装の考え方を補足した。付録の参照実装は、実際に実行して出力を確認したものである。なお、等幅フォントで表示できない一部の記号（チェックマークなど）は、意味を変えずに置き換えた。

1.3　本作のコードの特徴

本作では、日本政府の機密を含むデータを扱うため、**データの匿名化**（付録 A.1）が実装の前提になる。また、3 つの危機の同時発生を測る**三重リー括弧**、政府の組織を対象とする**層ラプラシアン**、国家規模の**手術計画**と 15 年間の**シンプレクティック積分**が登場する。

1.4　推奨ライブラリ

NumPy、SciPy、PyTorch、torchdiffeq、scikit-learn、EconML、SHAP、ripser / persim、Gambit（pygambit）を前提とする。ZKML には EZKL などのツールを使う。

第2章　多様体の構築

2.1　命題 1.1 の検証 ― 第 3 回試行（合格）

出典：本編第 3 章

背景　第 3 章「命題 1.1 ― 国家版」。省庁ごとの「データの島」をつなぐ橋（産業連関表など）と、マクロ経済サイクルの位相角を加え、3 条件をすべて満たした。

コード

```
# ===== 命題 1.1 第 3 回試行 =====
mse = 0.0052
print(f"条件 1：MSE = {mse:.4f} → 合格 OK")
knn = 0.918
print(f"条件 2：KNN 保存率 = {knn:.3f} → 合格 OK")
min_sv = 0.0027
print(f"条件 3：最小特異値 = {min_sv:.4f} → 合格 OK")
print("\n★ 経済安全保障多様体 ―― 20 次元微分可能多様体 ―― 構築完了 ★")
```

解説　再構成誤差（MSE）、近傍関係の保存率（KNN 保存率）、ヤコビ行列の最小特異値の 3 条件で、20 次元の多様体が幾何学的解析に使えることを確かめている。

実装上の注意

掲載コードは本編の数値を代入して判定結果を表示しているだけで、計算は省略されている。KNN 保存率は「データ空間で近い k 個の点のうち、潜在空間でも近い k 個に入っている割合」として定義を明記し、最小特異値は多数のサンプル点での分布（下側の分位点）で評価する。月次データだけでは約 180 時点しかないため、本編のように週次・日次の指標を軸に時点を増やし、補間した値には補間であることの印を付けて管理する。

参照　補助文書 02 第 2 章。コンパニオン Vol.1 第 5 章。

2.2　データの匿名化（本編第 1〜2 章の設計）

出典：本編第 1〜2 章（コードは付録 A.1）

背景　第 1〜2 章。警察庁警備局などの生データは、国家情報局の中で月次の集計指標にまとめてから多様体に入れる、という設計が示され、警備局のサイバー班が 3 週間かけて検証した。本編にコードは登場しないため、本書で参照実装を示す。

コード

```
# 付録 A.1 の aggregate_monthly() を参照
```

解説　個別の記録を月次・区分ごとの件数にまとめ、件数の少ない区分を伏せ、ラプラス分布のノイズを加えて差分プライバシーを付ける。VAE のデコーダが復元できるのは、この集計指標までとなる。

実装上の注意

VAE そのものは個別の事案を守る仕組みではない（デコーダが入力を復元するため）。守りは入力の段階で講じ、安全性は、デコーダの出力から個別の事案を推定する攻撃や、メンバーシップ推論攻撃を実際に試して確かめる。ノイズの強さ（ε）と集計の精度はトレードオフになるので、ε の値と、その根拠を記録しておく。

参照　補助文書 02 第 3 章。補助文書 04 第 13 節。

第3章　リー微分とリー括弧

3.1　台湾海峡封鎖演習のリー微分

出典：本編第 5 章

背景　第 5 章「台湾海峡」。封鎖演習を多様体の上のベクトル場として表し、多様体がどれだけ変形するかを測る。

コード

```
# ===== 台湾海峡封鎖演習のリー微分 =====
def X_taiwan_blockade(z):
    """台湾海峡封鎖演習を表すベクトル場。"""
    dx = np.zeros_like(z)
    dx[0] = -0.8    # 半導体供給（TSMC）が直接脅威
    dx[5] = -0.5    # エネルギー（海峡通過 LNG タンカー）
    dx[6] = -0.3    # 海底ケーブル（台湾周辺経路）
    dx[9] = +0.6    # 防衛産業の需要が急増（有事対応）
    dx[12] = +0.7   # 地政学リスク指数が急上昇
    return dx

L_taiwan = compute_lie_derivative(vae.decoder, z_current, X_taiwan_blockade)
print(f"台湾封鎖演習 リー微分トレース：{np.trace(L_taiwan):.2f}")
```

解説　半導体、エネルギー、海底ケーブル、防衛産業、地政学リスクの各次元に「押す向き」を与え、リー微分のトレースで変形の大きさを要約している（本編では 8.94）。

実装上の注意

掲載のベクトル場は状態 z によらない定数である。定数のベクトル場どうしのリー括弧は常に 0 になるため、次節以降で相互作用を測るには、効果の強さが状態に依存する形（たとえば中国依存度が高いほど打撃が大きい）にする必要がある（3 作目の補助文書 03 付録 B.2）。次元の番号と意味の対応は、データ辞書とあわせて管理する。

参照　補助文書 02 第 6.1 節。コンパニオン Vol.1 第 4 章。

3.2　リー括弧 — 台湾封鎖とレアアース停止

出典：本編第 6 章

背景　第 6 章「レアアースの沈黙」。堂本が指摘した「二正面作戦」を、リー括弧で確かめる。

コード

```
bracket_taiwan_rare = lie_bracket(X_taiwan_blockade, X_rare_earth_cutoff, z_current)
print(f"リー括弧 [台湾封鎖, レアアース停止] = {np.linalg.norm(bracket_taiwan_rare):.2f}")
```

解説　2 つの危機のリー括弧のノルム（本編では 6.41）が大きいほど、組み合わせの効果が個別の効果の和から外れる。

実装上の注意

リー括弧の計算には、ベクトル場のヤコビ行列が必要である。自動微分で $[X, Y] = J_Y X - J_X Y$ を計算する。ノルムの大きさは座標の尺度に依存するので、引き戻し計量 g で測ったノルム $\sqrt{v^\top g\, v}$ で比べるのが GI 理論の考え方に合う。

参照　補助文書 02 第 6.2 節。参考文献 [14]。

3.3　三重リー括弧 — 台湾 × レアアース × 穀物

出典：本編第 7 章

背景　第 7 章「穀物の人質」。中国の 2 つの圧力に、ロシアの穀物封鎖が重なる構造を測る。

コード

```
# 三重リー括弧：台湾 × レアアース × 穀物
bracket_rare_grain = lambda z: lie_bracket(X_rare_earth_cutoff, X_grain_blockade, z)  #
↪  内側の括弧も各点で計算する
triple_bracket = lie_bracket(X_taiwan_blockade, bracket_rare_grain, z_current)
print(f"三重リー括弧 [台湾, [レアアース, 穀物]] = {np.linalg.norm(triple_bracket):.2f}")
```

解説　内側の括弧 $[X_{\text{レアアース}}, X_{\text{穀物}}]$ を各点で計算する関数として定め、外側で台湾封鎖との括弧をとっている（本編では 8.73）。

実装上の注意

内側の括弧は、ある 1 点で計算した値（定数）ではなく、各点 z で計算し直す関数として外側の括弧に渡している。定数として渡すと、外側の括弧で内側の括弧の微分が 0 になり、正しい値にならないためである。自動微分を入れ子で使うので、`create_graph=True` を指定する。ヤコビ恒等式 $[X,[Y,Z]]+[Y,[Z,X]]+[Z,[X,Y]]=0$ で計算の正しさを検算できる。

参照　補助文書 02 第 6.3 節。

第4章　パーシステント・ホモロジーとゲーム理論

4.1　危機の点の集まりの形

出典：本編第 9 章

背景　第 9 章「パーシステント・ホモロジー ― 国家版」。4 つの主な危機に 22 の関連事象を加えて、形を調べる。

コード

```
from ripser import ripser
crisis_points = np.array([
    z_taiwan_blockade,     # ① 台湾海峡封鎖演習
    z_rare_earth_cutoff,   # ② レアアース停止
    z_grain_blockade,      # ③ 穀物封鎖
    z_cable_intrusion,     # ④ 海底ケーブルへの工作
] + related_events)        # 関連する 22 の小さな事象を追加
result = ripser(crisis_points, maxdim=1)
```

解説　ripser で連結成分（H_0）と輪（H_1）を計算し、台湾 → レアアース → 海底ケーブル → 台湾のループ（持続性 0.82）を見つけている。

実装上の注意

潜在座標をそのまま渡すと、点の間の距離はユークリッド距離になる。多様体の上の測地線距離の行列を作り、`ripser(D, distance_matrix=True)` として渡すのが望ましい。点の位置に小さな揺らぎを加えて計算を繰り返し、ループの持続性が安定しているかを確かめる。

参照　補助文書 02 第 8 章。

4.2　国家間のナッシュ均衡

出典：本編第 10 章

背景　第 10 章「3 つの国旗、2 つの同盟」。中国、ロシア、日本と QUAD・EU・英国の戦略を整理し、均衡を求める。

コード

```
players = {
    'China': {
        'objective': '台湾統一の環境整備＋経済覇権の確立',
        'strategies': ['軍事圧力維持', 'レアアース武器化', 'CIPS 拡大'],
    },
    'Russia': {
        'objective': 'NATO 弱体化＋資源支配力の維持',
        'strategies': ['穀物武器化', 'エネルギー武器化', 'サイバー攻撃'],
    },
    'Japan_QUAD': {
        'objective': '経済主権の維持＋同盟の強化',
        'strategies': [
            'S1_status_quo（現状維持＝個別対応）',
            'S2_surgery（産業構造改革＝脱中国依存）',
            'S3_alliance_deepening（QUAD＋EU＋英国の連携強化）',
            'S4_surgery_plus_alliance（手術＋同盟の複合戦略）',
        ],
    },
}
nash = compute_nash_equilibria(players, payoff_matrix, manifold)
```

解説　各プレイヤーの目的と戦略を列挙し、多様体から推定した利得の行列から均衡を求めている。本編では、日本の最適な応答が S4（手術と同盟の組み合わせ）となった。

実装上の注意

compute_nash_equilibria は本編では省略されている。3 人以上の有限ゲームの均衡は Gambit（pygambit）で計算できる（nashpy は 2 人ゲーム向け）。利得は推定値なので、利得を少しずつ変えて均衡が変わらないかを確かめる感度分析が欠かせない。均衡が複数ある場合は、どれを採用するかの基準を明記する。

参照　補助文書 02 第 9 章。コンパニオン Vol.2 第 23 章。

第5章　層ラプラシアン

5.1　政府の組織の層ラプラシアン

出典：本編第 11 章

背景　第 11 章「霞が関のモグラ」。職員データ、アクセス記録、通信経路の記録から組織の層を作り、不整合の大きい職員を探す。

コード

```
sheaf = construct_government_sheaf(
    personnel_data=nss_nia_personnel,
    access_logs=secure_environment_access_logs,
    data_flow_logs=encrypted_channel_logs
)
L = sheaf.laplacian()

x = sheaf.get_global_data_vector()  # 組織全体のデータを 1 つのベクトルに並べる
Lx = L @ x                          # 層ラプラシアンを一度だけ作用させる
for person in all_cleared_personnel:
    inconsistency = np.linalg.norm(sheaf.block(Lx, person))  # その職員に対応するブロックの大きさ
    if inconsistency > threshold:
        print(f"不整合検知：{person.name}（{person.org}）スコア={inconsistency:.2f}")
```

解説　職員（頂点）ごとにアクセスできるデータの範囲を、つながり（辺）ごとにデータの流れと権限の整合の規則を割り当て、層ラプラシアン L で食い違いを測る。

実装上の注意

組織全体のデータを 1 つのベクトルに並べて層ラプラシアンを一度だけ作用させ、その結果のうち各職員に対応するブロックの大きさをスコアにしている。職員一人分のデータに組織全体の層ラプラシアンを掛けると、行列とベクトルの大きさが合わないためである（最小限の参照実装は付録 A.2）。閾値は正常な期間のスコアの分布から決める。スコアは「調べるべき場所」を示すだけで不正の認定ではないため、法に基づく調査・捜査の手続きと組み合わせて運用する。

参照　補助文書 02 第 12 章。参考文献 [1]〜[4]、[18]。コンパニオン Vol.2 第 18 章。

第6章　手術とシンプレクティック積分

6.1　国家規模の手術計画

出典：本編第 12 章

背景　第 12 章「手術理論 — 国家の産業構造を変える」。切断すべき依存と、貼り合わせる新しい供給源を整理する。

コード

```
surgery_plan_japan = {
    'cut': [
        'レアアースの中国依存（60%→目標 15%以下）',
        '半導体の台湾 TSMC 集中（先端チップ 90%→目標 40%以下）',
        '原油の中東依存（87%→目標 60%以下）',
```

```
        '原薬の中国依存（抗生物質 90%→目標 30%以下）',
    ],
    'glue': [
        'レアアース：豪州・カナダ・アフリカ鉱山開発＋海底鉱物資源',
        '半導体：Rapidus（日本）＋Intel（米国）＋Samsung（韓国）の分散調達',
        '原油：米国シェール＋豪州＋再エネ比率引き上げ',
        '原薬：国内製造基盤の再構築＋インド・ベトナムへの分散',
    ],
    'cost': '18 兆円（10 年間）',
    'avoided_loss': '120 兆円（10 年間の累積損失回避額）',
}
```

解説　4 つの依存（レアアース、先端半導体、原油、原薬）の切断と、それぞれの分散先の貼り合わせ、費用と回避される損失を辞書としてまとめている。

実装上の注意

これは計画の記述であり、数学的な手術の計算ではない。計画の効果を数学的に確かめるには、計画を実行した後（またはそのシミュレーション）のデータで多様体を作り直し、パーシステント・ホモロジーで包囲ループが弱まるかを確認する。費用や回避額は、前提（為替、需要、各国の政策）とあわせて記録し、感度分析を行う。

参照　補助文書 02 第 10 章。参考文献 [5][6][7][15]。コンパニオン Vol.2 第 13 章。

6.2　15 年間の航海図

出典：本編第 12 章

背景　第 12 章。手術後の 15 年間（180 か月）の軌道を計算する。

コード

```
trajectory = simulate_symplectic(
    z_initial=z_post_surgery,
    time_horizon=180,   # 180 ヶ月 = 15 年
    dt=1.0,
    method='stormer_verlet',
)
print("15 年間シンプレクティック・シミュレーション結果：")
print(f"ハミルトニアン保存性：dH/H = {2.41e-8:.2e}")
```

解説　シュテルマー＝ヴェルレ法で 180 か月の軌道を計算し、ハミルトニアンの相対誤差（本編では 2.41×10^{-8}）で計算の安定性を確かめている。

実装上の注意

陽的なシュテルマー＝ヴェルレ法がシンプレクティックになるのは、ハミルトニアンが位置の部分と勢いの部分に分かれる場合である。勢いの部分が位置に依存する場合は、陰的な一般化リープフロッグ法を使う（3 作目の補助文書 03 付録 B.4）。相対誤差の分母はハミルトニアンの絶対値とする。誤差が小さいことは計算の正しさを示すだけで、15 年先の予測の確かさは示さないため、不確実性の幅を付けて報告する。

参照　補助文書 02 第 11 章。コンパニオン Vol.2 第 15 章。

第7章　ZKML

7.1　首相への最終証明

出典：本編第 17 章

背景　第 17 章「形式検証と ZKML」。分析が約束したモデルとデータから正しく計算されたことを、機密を明かさずに証明する。

コード

```
ZKML 証明：
  回路サイズ：5.1M ゲート（国家規模の多様体は最大級）
  証明生成：2 時間 17 分
  検証時間：1.8 秒
  結果：証明有効 OK
```

解説　回路規模 5.1M ゲートの証明を 2 時間 17 分で生成し、検証は 1.8 秒で済んだ。

実装上の注意

掲載内容は結果の表示である。実装には EZKL などのツールを使い、モデルを ONNX 形式で書き出して回路に変換する。計算は有限体の上で行うため、固定小数点への量子化の影響を評価する。パイプライン全体ではなく、デコーダ、因果効果の推定など重要な部分に絞って証明するのが現実的である。ZKML が保証するのは計算の正しさであり、データの解釈や政策判断の正しさではない。

参照　補助文書 02 第 13 章。参考文献 [16]。コンパニオン Vol.2 第 4・5 章。

付録 A　参照実装

本付録のコードは、本文の「実装上の注意」で述べた点を示すための最小限の例であり、本編の数値を再現するものではない。いずれも実際に実行し、出力を確認した。

A.1　集計・小さなセルの秘匿・差分プライバシー

```
import numpy as np

def aggregate_monthly(records, months, categories, k_min=5, epsilon=1.0, seed=0):
    """個別の記録（月, 区分）を月次・区分ごとの件数にまとめ、小さなセルを秘匿し、
    ラプラス・ノイズで差分プライバシーを付ける（1 件の増減で件数は最大 1 変わる）。"""
    rng = np.random.default_rng(seed)
    counts = np.zeros((len(months), len(categories)))
    for m, c in records:
        counts[months.index(m), categories.index(c)] += 1
    suppressed = counts < k_min                       # 小さなセルの秘匿
    noisy = counts + rng.laplace(0.0, 1.0 / epsilon, counts.shape)
    noisy[suppressed] = np.nan                        # 伏せた区分は欠測として扱う
    return np.round(noisy, 1), suppressed

months = ['2026-04', '2026-05', '2026-06']
cats = ['接触', '訪問', '通信']
table = {('2026-04', '接触'): 12, ('2026-04', '訪問'): 9, ('2026-04', '通信'): 3,
         ('2026-05', '接触'): 15, ('2026-05', '訪問'): 11, ('2026-05', '通信'): 8,
         ('2026-06', '接触'): 21, ('2026-06', '訪問'): 2,  ('2026-06', '通信'): 10}
records = [key for key, n in table.items() for _ in range(n)]   # 個別の記録（例）
agg, sup = aggregate_monthly(records, months, cats)
print(agg)
print('秘匿したセルの数:', int(sup.sum()))
```

実行結果（ノイズは乱数の種を固定して生成）：

```
[[12.3  8.4  nan]
 [11.6 12.   9.7]
 [21.2  nan 10.1]]
秘匿したセルの数: 2
```

件数が 5 未満の区分（2026 年 4 月の「通信」3 件、2026 年 6 月の「訪問」2 件）は伏せられ、それ以外の件数にはノイズが加わっている。多様体には、このような集計指標だけが入る。

A.2　層ラプラシアンと頂点ごとのスコア

```
import numpy as np

def sheaf_laplacian(n_nodes, stalk_dim, edges):
```

```
    """edges: [(u, v, F_u, F_v)]。F_u, F_v は頂点のデータを辺の上で比べる制限写像（行列）。"""
    rows = []
    for u, v, Fu, Fv in edges:
        r = np.zeros((Fu.shape[0], n_nodes * stalk_dim))
        r[:, u*stalk_dim:(u+1)*stalk_dim] = -Fu
        r[:, v*stalk_dim:(v+1)*stalk_dim] = Fv
        rows.append(r)
    delta = np.vstack(rows)          # 余境界作用素 δ
    return delta, delta.T @ delta    # 層ラプラシアン L = δ^T δ

def node_scores(L, x, n_nodes, stalk_dim):
    """全体のデータ x に L を一度だけ作用させ、頂点ごとのブロックの大きさをスコアにする。"""
    Lx = (L @ x).reshape(n_nodes, stalk_dim)
    return Lx, np.linalg.norm(Lx, axis=1)

# 3 部署の演習問題：A(営業), B(物流), C(経理)。各部署は（数量, 単価）を持つ
I = np.eye(2)
edges = [(0, 1, I, I), (1, 2, I, I), (2, 0, I, I)]   # 両端の記録は一致すべき
x = np.array([10, 50,  10, 50,  10, 80], float)
delta, L = sheaf_laplacian(3, 2, edges)
Lx, score = node_scores(L, x, 3, 2)
print("L x =", Lx.tolist())
print("スコア =", np.round(score, 1).tolist(), " 整合性半径 =", round(float(np.linalg.norm(delta @ x)), 2))
```

実行結果：

```
L x = [[0.0, -30.0], [0.0, -30.0], [0.0, 60.0]]
スコア = [30.0, 30.0, 60.0]  整合性半径 = 42.43
```

組織全体のデータを 1 つのベクトルに並べ、層ラプラシアンを一度だけ作用させて、頂点ごとのブロックの大きさをスコアにしている。補助文書 02 第 12.7 節の演習問題と同じ結果である。